UNIVERSITE CHOUAIB DOUKKALI
ECOLE NATIONALE DES SCIENCES APPLIQUEES D'EL JADIDA

جامعة شعيب الدكالي
Université Chouaib Doukkali

Cybersécurité & Confiance Numérique (CCN)

Algorithmique et programmation avancée avec Python

Pr. Fatima zahra SALMAM
salmam.f@ucd.ac.ma

python™

1

Partie III :

- ✓Conteneurs standard: listes, dictionnaires, tuples et sets
- ✓ Les fichiers
- ✓ Les modules

32

Les modules: définition

- Les modules sont des programmes Python qui contiennent des fonctions que l'on est amené à réutiliser souvent (on les appelle aussi bibliothèques ou librairies).
- De nombreux modules qui effectuent une quantité phénoménale de tâches sont prédéfinis.
- Pour cette raison, prenez toujours le réflexe de vérifier si une partie du code que vous souhaitez écrire n'existe pas déjà sous forme de module.
- La plupart de ces modules sont déjà installés dans les versions standards de Python. Vous pouvez accéder à une documentation exhaustive sur le site de Python.

33

Les modules: Importation de modules

- Dans les parties précédentes de ce cours, nous avons rencontré la notion de module plusieurs fois. Par exemple, lorsque nous avons voulu tirer un nombre aléatoire :

```
>>> import random
>>> random . randint (0 , 10)
    4
```

- L'instruction import donne accès à toutes les fonctions du module **random**.
- La fonction **randint(0, 10)** du module **random** renvoie un nombre entier tiré aléatoirement entre 0 inclus et 10 inclus.

34

Les modules: Importation de modules

- L'utilisation de la syntaxe import module permet d'importer tout une série de fonctions organisées par «thèmes ».
- Par exemple, les fonctions gérant les nombres aléatoires avec **random** et les fonctions mathématiques avec **math**.
- Python possède de nombreux autres modules internes (c'est-à-dire présent de base lorsqu'on installe Python).
- Une méthode est une fonction un peu particulière :
  - Elle est liée à un objet par un point;
  - En général, elle agissait sur ou utilisait l'objet auquel elle était liée.

35

Les modules: Importation de modules

- Par exemple, la méthode **.format()** dans l'instruction **"{}".format(3.14)** utilise l'objet chaîne de caractères **"{}"** (auquel elle est liée) pour finalement renvoyer une autre chaîne de caractères **"3.14"**.
- Il existe un autre moyen d'importer une ou plusieurs fonctions d'un module :

```
>>> from random import randint
>>> randint (0 , 10)
    7
```

- À l'aide du mot-clé **from**, on peut importer une fonction spécifique d'un module donné. Remarquez bien qu'il est inutile de répéter le nom du module dans ce cas, seul le nom de la fonction en question est requis. On peut également importer toutes les fonctions d'un module :

```
>>> from random import *
```


36

Les modules: Importation de modules

- L'instruction from random import * importe toutes les fonctions du module **random**. On peut ainsi utiliser toutes ses fonctions directement, comme par exemple **shuffle()** qui permute une liste aléatoirement.
- Il est également possible de définir un alias (un nom plus court) pour un module :

```
>>> import random as rand
>>> rand.randint (0 , 10)
    6
```

- Dans cet exemple, les fonctions du module random sont accessibles via l'alias rand.

37

Les modules: Importation de modules

- Enfin, pour vider de la mémoire un module déjà chargé, on peut utiliser l'instruction del :

```
>>> import random
>>> random . randint (0 ,10)
      2
>>> del random
>>> random . randint (0 ,10)
      Traceback ( most recent call last ):
      File " < stdin >" , line 1 , in ?
      NameError : name 'random ' is not defined
```

- On constate alors qu'un rappel d'une fonction du module **random** après l'avoir vidé de la mémoire retourne un message d'erreur.

38

Les modules: Importation de modules

- Pour obtenir de l'aide sur un module, il suffit d'utiliser la commande help() :

```
>>> import random
>>> help(random)
```

- La commande help() est une commande plus générale permettant d'avoir de l'aide sur n'importe quel objet chargé en mémoire

```
>>> t = [1, 2, 3]
>>> help(t)
Help on list object:

class list(object)
 |  list() -> new list
 |  list(sequence) -> new list initialized from sequence's items
 |
 |  Methods defined here:
 |
 |  __add__(...)
 |      x.__add__(y) <==> x+y
 |
...
```


39

Les modules: Importation de modules

- Pour connaître d'un seul coup d'œil toutes les méthodes ou variables associées à un objet, utilisez la fonction **dir()** :

```
>>> import random
>>> dir(random)
['BPF', 'LOG4', 'NV_MAGICCONST', 'RECIP_BPF', 'Random', 'SG_MAGICCONST',
'SystemRandom', 'TWOPI', 'WichmannHill', '_BuiltinMethodType', '_MethodT
ype', '__all__', '__builtins__', '__doc__', '__file__', '__name__', '_ac
os', '_ceil', '_cos', '_e', '_exp', '_hexlify', '_inst', '_log', '_pi',
'_random', '_sin', '_sqrt', '_test', '_test_generator', '_urandom', '_wa
rn', 'betavariate', 'choice', 'expovariate', 'gammavariate', 'gauss', 'g
etrandbits', 'getstate', 'jumpahead', 'lognormvariate', 'normalvariate',
 'paretovariate', 'randint', 'random', 'randrange', 'sample', 'seed', 's
```


40

Les modules: Module sys (passage d'arguments)

- Le module **sys** contient des fonctions et des variables spécifiques à l'interpréteur Python lui-même. Ce module est particulièrement intéressant pour récupérer les arguments passés à un script Python lorsque celui-ci est appelé en ligne de commande.
- Par exemple, si nous créons un court script que l'on enregistrera sous le nom test.py :

```
1 import sys
2 print(sys.argv)
```

- Ensuite, dans un shell, nous exécutons le script test.py suivi de plusieurs arguments. Par exemple :

```
1 $ python test.py salut girafe 42
2 ['test.py', 'salut', 'girafe', '42']
```


41

Les modules: Module sys (passage d'arguments)

- Ensuite, dans un shell, nous exécutons le script test.py suivi de plusieurs arguments. Par exemple :

```
1 $ python test.py salut girafe 42
2 ['test.py', 'salut', 'girafe', '42']
```

- Le script affiche le contenu de la variable sys.argv. Cette variable est une liste qui contient tous les arguments de la ligne de commande, y compris le nom du script lui-même qu'on retrouve comme premier élément de cette liste dans sys.argv[0].
- On peut donc accéder à chacun des arguments du script avec sys.argv[1], sys.argv[2]. . .

42

Les modules: Module sys (passage d'arguments)

- La fonction **sys.exit()** est utile pour quitter un script Python. On peut donner un argument à cette fonction (en général une chaîne de caractères) qui sera renvoyé au moment où Python quittera le script.
- Par exemple, si vous attendez au moins un argument en ligne de commande, vous pouvez renvoyer un message pour indiquer à l'utilisateur ce que le script attend comme argument :

```
import sys

if len(sys.argv) != 2:
    sys.exit("ERREUR : il faut exactement un argument.")

print("Argument vaut : {}".format(sys.argv[1]))
```


43

Les modules: Module sys (passage d'arguments)

- Puis on l'exécute sans argument :

```
$ python test.py
ERREUR : il faut exactement un argument.
```

- Et avec argument:

```
$ python test.py 42
Argument vaut : 42
```

- Notez qu'ici on vérifie que le script possède deux arguments car le nom du script lui-même compte pour un argument (le tout premier).
- L'intérêt de récupérer des arguments passés dans la ligne de commande à l'appel du script est de pouvoir ensuite les utiliser dans le script.

44

Les modules: Module sys (passage d'arguments)

- Voici à titre d'exemple un script **compte_lignes.py** qui va prendre comme argument le nom d'un fichier puis afficher le nombre de lignes qu'il contient.

```
import sys

if len(sys.argv) != 2:
    sys.exit("ERREUR : il faut exactement un argument.")

nom_fichier = sys.argv[1]
taille = 0
with open(nom_fichier, "r") as f_in:
    taille = len(f_in.readlines())

print("{} contient {} lignes.".format(nom_fichier, taille))
```

- Supposons que dans le même répertoire, nous avons un fichier **zoo1.txt** dont voici le contenu :

```
girafe
tigre
singe
souris
```


45

Les modules: Module sys (passage d'arguments)

- et le fichier zoo2.txt qui contient :

```
poisson
abeille
chat
```

- Si on utilise maintenant le script **compte_lignes.py** :

```
$ python compte_lignes.py
ERREUR : il faut exactement un argument.
$ python compte_lignes.py zoo1.txt
zoo1.txt contient 4 lignes.
$ python compte_lignes.py zoo2.txt
zoo2.txt contient 3 lignes.
```

- Notre script est donc capable de :
  - Vérifier si un argument lui est donné et si ce n'est pas le cas d'afficher un message d'erreur.
  - D'ouvrir le fichier dont le nom est fourni en argument, de compter puis d'afficher le nombre de lignes.
- Par contre, le script ne vérifie pas si le fichier existe bien :

```
$ python compte_lignes.py zoo3.txt
Traceback (most recent call last):
  File "compte_lignes.py", line 8, in <module>
    with open(nom_fichier, "r") as f_in:
FileNotFoundError: [Errno 2] No such file or directory: 'zoo3.txt'
```


46

Les modules: Module os (interaction avec le SE)

- Le module **os** gère l'interface avec le système d'exploitation.
- La fonction **os.path.exists()** est une fonction pratique de ce module qui vérifie la présence d'un fichier sur le disque dur.

```
>>> import sys
>>> import os
>>> if os.path.exists("toto.pdb"):
...     print("le fichier est présent")
... else:
...     sys.exit("le fichier est absent")
...
le fichier est absent
```

- Dans cet exemple, si le fichier n'existe pas sur le disque, on quitte le programme avec la fonction **exit()** du module **sys**.

47

Les modules: Module os (interaction avec le SE)

- Améliorez le script **compte_lignes.py** dont le code a été donné précédemment de façon à ce qu'il renvoie un message d'erreur si le fichier n'existe pas. Par exemple, si les fichiers **zoo1.txt** et **zoo2.txt** sont bien dans le répertoire courant, mais pas **zoo3.txt**

```
import sys

if len(sys.argv) != 2:
    sys.exit("ERREUR : il faut exactement un argument.")

nom_fichier = sys.argv[1]
taille = 0
with open(nom_fichier, "r") as f_in:
    taille = len(f_in.readlines())

print("{} contient {} lignes.".format(nom_fichier, taille))
```


48

Les modules: Exercice

- Utilisez l'interpréteur Python pour affichez sur la même ligne les nombres de 10 à 20 (inclus) ainsi que leur racine carrée avec 3 décimales. Utilisez pour cela le module math avec la fonction sqrt(). Exemple :

```
10 3.162
11 3.317
12 3.464
13 3.606
[...]
```


49

Les modules: Exercice

- Affichez les nombres de 1 à 10 avec 1 seconde d'intervalle. Utilisez pour cela le module **time** et sa fonction **sleep().**
- Créez un script qui prend le nombre de seconde comme argument, puis exécutez-le dans un shell.

50

Les modules: Création de modules

- En Python, la création d'un module est très simple. Il suffit d'écrire un ensemble de fonctions (et/ou de constantes) dans un fichier, puis d'enregistrer ce dernier avec une extension **.py** (comme n'importe quel script Python).
- Par exemple, nous allons créer un module simple que nous enregistrerons sous le nom **message.py**

```
"""Module inutile qui affiche des messages :-)."""

DATE = 16092008


def bonjour(nom):
    """Dit Bonjour."""
    return "Bonjour " + nom


def ciao(nom):
    """Dit Ciao."""
    return "Ciao " + nom


def hello(nom):
    """Dit Hello."""
    return "Hello " + nom
```


Les modules: Utilisation de son propre module

- Pour appeler une fonction ou une variable de ce module, il faut que le fichier **message.py** soit dans le répertoire courant (dans lequel on travaille) ou bien dans un répertoire listé par la variable d'environnement PYTHONPATH de votre système d'exploitation.
- Ensuite, il suffit d'importer le module et toutes ses fonctions (et constantes) vous sont alors accessibles.
- Avec Windows, depuis un shell PowerShell, il faut taper la commande suivante :

  $env:PYTHONPATH += ";C:\chemin\vers\mon\super\module"

- Une fois cette manipulation effectuée, vous pouvez contrôler que le chemin vers le répertoire contenant vos modules a bien été ajouté à la variable d'environnement PYTHONPATH : : echo $env:PYTHONPATH

Les modules: Utilisation de son propre module

- Le chargement du module se fait avec la commande **import message**.
- Notez que le fichier est bien enregistré avec une extension .py et pourtant on ne la précise pas lorsqu'on importe le module.
- Ensuite, on peut utiliser les fonctions comme avec un module classique.

```
>>> import message
>>> message.hello("Joe")
'Hello Joe'
>>> message.ciao("Bill")
'Ciao Bill'
>>> message.bonjour("Monsieur")
'Bonjour Monsieur'
>>> message.DATE
16092008
```

Remarque:

- La première fois qu'un module est importé, Python crée un répertoire nommé _ _pycache_ _ contenant un fichier avec une extension .pyc qui contient le bytecode (le code précompilé du module).

Les modules: Utilisation de son propre module

Les docstrings:

- Lorsqu'on écrit un module, il est important de créer de la documentation pour expliquer ce que fait le module et comment utiliser chaque fonction.
- Les chaînes de caractères entre triple guillemets situées en début du module et de chaque fonction on les appelle docstrings (« chaînes de documentation » en français).
- Ces docstrings permettent notamment de fournir de l'aide lorsqu'on invoque la commande help() :

```
>>> help(message)

Help on module message:

NAME
    message - Module inutile qui affiche des messages :-).

FUNCTIONS
    bonjour(nom)
        Dit Bonjour.

    ciao(nom)
        Dit Ciao.

    hello(nom)
        Dit Hello.

DATA
    DATE = 16092008

FILE
    /home/pierre/message.py
```

Les modules: Utilisation de son propre module

Les docstrings:

- on peut aussi appeler l'aide pour une seule fonction :

```
>>> help(message.ciao)

Help on function ciao in module message:

ciao(nom)
    Dit Ciao.
```

- En résumé, les docstrings sont destinés aux utilisateurs du module. Leur but est différent des commentaires qui, eux, sont destinés à celui qui lit le code (pour en comprendre les subtilités).
- Une bonne docstring de fonction doit contenir tout ce dont un utilisateur a besoin pour utiliser cette fonction. Une liste minimale et non exhaustive serait :
  - ce que fait la fonction,
  - ce qu'elle prend en argument,
  - ce qu'elle renvoie.

55

Les modules: Utilisation de son propre module

Visibilité des fonctions dans un module :

- La visibilité des fonctions au sein des modules suit des règles simples :
  - Les fonctions dans un même module peuvent s'appeler les unes les autres.
  - Les fonctions dans un module peuvent appeler des fonctions situées dans un autre module s'il a été préalablement importé.
  - Par exemple, si la commande **import autremodule** est utilisée dans un module, il est possible d'appeler une fonction avec **autremodule.fonction().**

56

Les modules: Utilisation de son propre module

Module ou script ?

- Le module message ne contient que des fonctions et une constante. Si on l'exécutait comme un script classique, cela n'afficherait rien.

```
$ python message.py
$
```

- Cela s'explique par l'absence de programme principal, c'est-à-dire, de lignes de code que l'interpréteur exécute lorsqu'on lance le script.
- À l'inverse, que se passe-t-il alors si on importe un script en tant que module alors qu'il contient un programme principal avec des lignes de code ?
- Prenons par exemple le script message2.py suivant :

```
"""Script de test."""


def bonjour(nom):
    """Dit Bonjour."""
    return "Bonjour " + nom


# programme principal
print(bonjour("Joe"))
```

- Si on l'importe dans l'interpréteur, on obtient :

```
>>> import message2
Bonjour Joe
```


Les modules: Utilisation de son propre module

Module ou script ?

- Ceci n'est pas le comportement voulu pour un module car on n'attend pas d'affichage particulier (par exemple la commande import math n'affiche rien dans l'interpréteur).
- Afin de pouvoir utiliser un code Python en tant que module ou en tant que script, il faut utiliser la structure suivante :

```
"""Script de test."""


def bonjour(nom):
    """Dit Bonjour."""
    return "Bonjour " + nom


if __name__ == "__main__":
    print(bonjour("Joe"))
```

- À la ligne 9, l'instruction if _ _name_ _ == "_ _main_ _": indique à Python :
  - Si le programme message2.py est exécuté en tant que script dans un shell, le résultat du test if sera alors True et le bloc d'instructions correspondant (ligne 10) sera exécuté.

```
$ python message2.py
Bonjour Joe
```

  - Si le programme message2.py est importé en tant que module, le résultat du test if sera alors False (et le bloc d'instructions correspondant ne sera pas exécuté) :

UNIVERSITE CHOUAIB DOUKKALI
ECOLE NATIONALE DES SCIENCES APPLIQUEES D'EL JADIDA

جامعة شعيب الدكالي
Université Chouaib Doukkali

Cybersécurité & Confiance Numérique (CCN)

Algorithmique et programmation avancée avec Python

Pr. Fatima zahra SALMAM
salmam.f@ucd.ac.ma

python™

1

Partie IV :

- ✓ Les exceptions
- ✓ Les expressions régulières

2

Les exceptions python

Les langages de programmation comme Python contiennent un système de gestion des exceptions.

- **Qu'est-ce qu'une exception ?**
  - Une exception est définie comme une anomalie de l'exécution d'un programme requérant une action spéciale, en général l'arrêt de l'exécution.
  - Le plus souvent une exception correspond à une erreur que Python rencontre lorsqu'il tente d'exécuter les lignes de code qu'on lui soumet.
  - Par exemple, un problème de syntaxe, une variable ou objet qui prend une valeur aberrante (par exemple diviser par 0, parcourir une liste au-delà du nombre d'éléments, etc.).
- **Exemple:**

```
>>> 5/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
```


Les exceptions python

- Le système de gestion des exceptions évite que votre programme « plante » en prévoyant vous-même les sources d'erreurs éventuelles.
- Voici un exemple dans lequel on demande à l'utilisateur d'entrer un nombre entier, puis on affiche ce nombre.

```
>>> nb = int(input("Entrez un nombre entier : "))
Entrez un nombre entier : 23
>>> print(nb)
23
```

- La fonction **input()** demande à l'utilisateur de saisir une chaîne de caractères. Cette chaîne de caractères est ensuite transformée en nombre entier avec la fonction **int().**
- Si l'utilisateur ne rentre pas un nombre, voici ce qui se passe :

```
>>> nb = int(input("Entrez un nombre entier : "))
Entrez un nombre entier : ATCG
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'ATCG'
```

Les exceptions python

- L'erreur provient de la fonction **int()** qui n'a pas pu convertir la chaîne de caractères "ATCG" en nombre entier, ce qui est parfaitement normal. En termes plus techniques, on dira que « Python a levé une exception de type ValueError ».
- Il y a de nombreux types d'exceptions différents! Le nom de l'exception apparaît toujours comme le premier mot de la dernière ligne du message d'erreur.
- Si nous lancions ces lignes de code sous forme de script (du style python script.py), cet exemple conduirait à l'arrêt de l'exécution du programme.
- Le jeu d'instructions **try / except** permet de tester l'exécution d'une commande et d'intervenir en cas de levée d'exception.

```
>>> try:
...     nb = int(input("Entrez un nombre entier : "))
... except:
...     print("Vous n'avez pas entré un nombre entier !")
...
Entrez un nombre entier : ATCG
Vous n'avez pas entré un nombre entier !
```


5

Les exceptions python

- Dans l'exemple précèdent, l'exception levée par la fonction **int()** (qui ne peut pas convertir "ATCG" en nombre entier) est interceptée et déclenche l'affichage du message d'avertissement.
- On peut ainsi redemander sans cesse un nombre entier à l'utilisateur, jusqu'à ce que celui-ci en rentre bien un.

```
>>> while True:
...     try:
...             nb = int(input("Entrez un nombre entier : "))
...             print("Le nombre est", nb)
...             break
...     except:
...             print("Vous n'avez pas entré un nombre entier !")
...             print("Essayez encore")
...
Entrez un nombre entier : ATCG
Vous n'avez pas entré un nombre entier !
Essayez encore
Entrez un nombre entier : toto
Vous n'avez pas entré un nombre entier !
Essayez encore
Entrez un nombre entier : 3.2
Vous n'avez pas entré un nombre entier !
Essayez encore
Entrez un nombre entier : 55
Le nombre est 55
```

- Notez que dans cet exemple, l'instruction **while True** est une boucle infinie car la condition **True** est **toujours vérifiée**.
- L'arrêt de cette boucle est alors forcé par la commande **break** lorsque l'utilisateur a effectivement entré un nombre entier.

6

Les exceptions python

- La gestion des exceptions est très utile dès lors que des données extérieures entrent dans un programme Python, que ce soit directement par l'utilisateur (avec la fonction input()) ou par des fichiers. Cela est fondamental si vous distribuez votre code à la communauté : si les utilisateurs ne connaissent pas Python, **un message comme** Vous n'avez pas entré un nombre entier! **reste plus clair que** ValueError: invalid literal for int() with base 10: 'ATCG'

- Vous pouvez par exemple vérifier qu'un fichier a bien été ouvert.

```
>>> nom = "toto.pdb"
>>> try:
...     with open(nom, "r") as fichier:
...         for ligne in fichier:
...             print(ligne)
... except:
...     print("Impossible d'ouvrir le fichier", nom)
```

- Si une erreur est déclenchée, c'est sans doute que le fichier n'existe pas à l'emplacement indiqué sur le disque ou que vous n'avez pas les droits pour le lire.

7

Les exceptions python

Cibler les erreurs:

- Il est également possible de spécifier le type d'erreur à gérer. Le premier exemple que nous avons étudié peut s'écrire :

```
>>> try:
...     nb = int(input("Entrez un nombre entier : "))
... except ValueError:
...     print("Vous n'avez pas entré un nombre entier !")
...
Entrez un nombre entier : ATCG
Vous n'avez pas entré un nombre entier !
```

- Ici, on intercepte une exception de type **ValueError**, ce qui correspond bien à un problème de **conversion avec int().**

- Attention, si vous précisez le type d'exception comme **ValueError**, le except ValueError **n'empêchera pas la levée d'une autre exception**.

8

Les exceptions python

```
>>> try:
...     nb = int(variable)
... except ValueError:
...     print("Vous n'avez pas entré un nombre entier !")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
NameError: name 'variable' is not defined
```

- Ici l'exception levée est de type **NameError** car variable n'existe pas. Alors que si vous mettez **except tout court**, **cela intercepte n'importe quelle exception.**

```
>>> try:
...     nb = int(variable)
... except:
...     print("Vous n'avez pas entré un nombre entier !")
...
Vous n'avez pas entré un nombre entier !
>>>
```

- Vous voyez qu'ici cela pose un nouveau problème : le message d'erreur ne correspond pas à l'exception levée !

9

Les exceptions python

```
>>> try:
...     nb = int(variable)
... except ValueError:
...     print("Vous n'avez pas entré un nombre entier !")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
NameError: name 'variable' is not defined
```

- Ici l'exception levée est de type **NameError** car variable n'existe pas. Alors que si vous mettez **except tout court**, **cela intercepte n'importe quelle exception.**

```
>>> try:
...     nb = int(variable)
... except:
...     print("Vous n'avez pas entré un nombre entier !")
...
Vous n'avez pas entré un nombre entier !
>>>
```

- Vous voyez qu'ici cela pose un nouveau problème : le message d'erreur ne correspond pas à l'exception levée !

10

Les exceptions python

- Il est possible en python d'exécuter des instructions en fonction du type d'erreur. Par exemple si la valeur est 0 on aimerait afficher un message et si le dénominateur est du texte on aimerait pouvoir afficher un autre message à l'utilisateur.

```
>>> v = 0
>>> w = 5
>>> try:
... w/v
... print("Ok pas erreur")
... except TypeError:
... print("Merci d'utiliser des chiffres")
... except ZeroDivisionError:
... print("Merci de ne pas utiliser le 0")
...
Merci de ne pas utiliser le 0
```


11

Les exceptions python

- et dans le cas où la variable v vaut " olivier ":

```
>>> v = "olivier"
>>> w = 5
>>> try:
... w/v
... print("Ok pas erreur")
... except TypeError:
... print("Merci d'utiliser des chiffres")
... except ZeroDivisionError:
... print("Merci de ne pas utiliser le 0")
...
Merci d'utiliser des chiffres
```

- On utilise le mot clé finally pour exécuter des instructions quel que soit les erreurs générées ou non (et même si il y a présence d'un return). Dans tous les cas les instructions placées dans finally seront executées.

```
try:
     pass
except:
     pass
finally:
     print("Execution")
```


12

Les exceptions python

Remarques:

- Nous vous conseillons vivement de toujours préciser le type d'exception dans vos except.
- Cela évite d'intercepter une exception que vous n'aviez pas prévue.
- Il est possible d'intercepter plusieurs types d'exceptions en passant un tuple à except, par exemple : except (Exception1, Exception2).
- Par ailleurs, ne mettez pas trop de lignes dans le bloc du try. Dans un tel cas, il peut être très pénible de trouver la ligne qui a conduit à l'exécution du except.
- Pire encore, il se peut que des lignes que vous aviez prévues ne soient pas exécutées !
- Donc gardez des choses simples dans un premier temps, comme par exemple tester les conversions de type ou vérifier qu'un fichier existe bien et que vous pouvez l'ouvrir.

- Il existe de nombreux types d'exception comme RuntimeError, TypeError, NameError, IOError, etc. Vous pouvez aller voir la liste complète sur le site de Python.

Les exceptions python: Exercice

- Ecrivez un programme qui demande à l'utilisateur de saisir un nombre et affiche son inverse. Gérer toutes les exceptions possibles.

Partie IV :

- ✓ Les exceptions
- ✓ Les expressions régulières

15

Les expressions régulières

- Les expressions régulières (rationnelles) sont utilisées dans quasiment tous les langages. C'est un outil très puissant qui permet de vérifier si le contenu d'une variable a la forme de ce que l'on attend.
- Par exemple si on récupère un numéro de téléphone, on s'attend à ce que la variable soit composée de nombres et d'espaces (ou de tiret) mais rien de plus.
- Les expressions régulières permettent non seulement de vous avertir d'un caractère non désiré mais également de supprimer/modifier tous ceux qui ne sont pas désirables.
- Concrètement, les expressions régulières vont nous permettre de vérifier la présence de certains caractères ou suites de caractères dans une expression afin de valider la forme de données envoyées par l'utilisateur via une fonction **input()** par exemple.

16

Les expressions régulières

Les bases

- On utilise des symboles qui ont une signification:

```
. ^ $ * + ? { } [ ] \ | ( )
```

```
.      Le point correspond à n'importe quel caractère.
^      Indique un commencement de segment mais signifie aussi "contraire de"
$      Fin de segment
[xy]   Une liste de segment possibble. Exemple [abc] équivaut à : a, b ou c
(x|y)  Indique un choix multiple type (ps|ump) équivaut à "ps" OU "UMP"
\d     le segment est composé uniquement de chiffre, ce qui équivaut à [0-9].
\D     le segment n'est pas composé de chiffre, ce qui équivaut à [^0-9].
\s     Un espace, ce qui équivaut à [ \t\n\r\f\v].
\S     Pas d'espace, ce qui équivaut à [^ \t\n\r\f\v].
\w     Présence alphanumérique, ce qui équivaut à [a-zA-Z0-9_].
\W     Pas de présence alphanumérique [^a-zA-Z0-9_].
\      Est un caractère d'échappement
```


17

Les expressions régulières

- Il est possible d'imposer le nombre d'occurrences avec la syntaxe suivante:

A{2} : on attend à ce que la lettre A (en majuscule) se répète 2 fois consécutives.

BA{1,9} : on attend à ce que le segment BA se répète de 1 à 9 fois consécutives.

BRA{,10} : on attend à ce que le segment BRA ne soit pas présent du tout ou présent jusqu'à 10 fois consécutives.

VO{1,} : on attend à ce que le segment VO soit présent au moins une fois.

Symbole	Nb Caractères attendus	Exemple	Cas possibles
?	0 ou 1	GR (.)?S	GRS, GROS, GRIS, GRAS
+	1 ou plus	GR (.)+S	GROS, GRIS, GRAS
*	0, 1 ou plus	GR (.)*S	GRS,GROS ,GRIIIS,GROlivierS

18

Les expressions régulières

- Python nous permet d'exploiter les expressions régulières via son module standard **re**.
- Nous allons donc utiliser des fonctions qui vont permettre d'effectuer des recherches de motifs dans des chaines, des remplacements, etc.
- Une expression régulière en soi n'est qu'un motif. Nous allons donc utiliser des fonctions qui vont permettre d'effectuer des recherches de motifs dans des chaines, des remplacements, etc.
- La fonction **compile()** compile un motif vers une expression rationnelle compilée qu'on va pouvoir utiliser avec les méthodes **match()** et **search()**.
- On utilisera **compile()** lorsqu'on voudra sauvegarder une expression régulière pour la réutiliser dans un programme.

19

Les expressions régulières

- La fonction **match()** prend également un schéma de recherche en premier argument et une chaine dans laquelle effectuer la recherche en deuxième argument et renvoie l'objet de correspondance trouvé si zéro ou plus caractères au début de la chaine correspondent à l'expression régulière ou None si la chaîne ne correspond pas au motif.

```
>>> import re
>>> p = re.compile('[a-z]+')
```

- Maintenant, vous pouvez tester des correspondances de la RE **[a-z]+** avec différentes chaînes.
- Une chaîne vide ne doit pas correspondre, puisque **+** indique « **une ou plusieurs occurrences** ». **match()** doit renvoyer **None** dans ce cas, ce qui fait que l'interpréteur n'affiche rien.

20

Les expressions régulières

Exemple :

```
>>> print(p.match(""))
None
```

Sur une chaîne qui correspond, par exemple **tempo**. Dans ce cas, **match()** renvoie un objet correspondance, vous pouvez ainsi stocker le résultat dans une variable pour une utilisation ultérieure.

```
>>> m = p.match('tempo')
>>> m
<re.Match object; span=(0, 5), match='tempo'>
```


21

Les expressions régulières

Vous pouvez interroger l'objet correspondance pour obtenir des informations sur la chaîne qui correspond.

Les instances d'objets correspondances possèdent plusieurs méthodes et attributs ; les plus importants sont :

Méthode/Attribut	Objectif
group()	Renvoie la chaîne de caractères correspondant à la RE
start()	Renvoie la position de début de la correspondance
end()	Renvoie la position de fin de la correspondance
span()	Renvoie un *n*-uplet contenant les positions (début, fin) de la correspondance

22

Les expressions régulières

Exemple:

```
>>> m.group()
'tempo’
>>> m.start(), m.end()
(0, 5)
>>> m.span()
(0, 5)
```


23

Les expressions régulières

- La fonction **search()** permet d’analyser une chaine passée en deuxième argument à la recherche du premier emplacement où l’expression régulière passé en premier argument trouve une correspondance, et renvoie l’objet de correspondance trouvé ou None si aucune position dans la chaîne ne valide le motif.
- En d'autres termes, la méthode **search()** d'un motif analyse toute la chaîne, afin de trouver une correspondance potentielle qui ne commence pas à zéro.

```
>>> print(p.match('::: message’))
None
>>> m = p.search('::: message’);
print(m) <re.Match object; span=(4, 11), match='message’>
>>> m.group()
'message’
>>> m.span()
(4, 11)
```


24

Les expressions régulières

- Dans les programmes réels, la façon de faire la plus courante consiste à stocker l'objet correspondance dans une variable, puis à vérifier s'il vaut None. Généralement, cela ressemble à ceci :

```
p = re.compile( ... )
m = p.match( 'string goes here' )
        if m:
                print('Match found: ', m.group())
        else:
                print('No match')
```


25

Les expressions régulières

- Il existe deux méthodes de motifs qui renvoient toutes les correspondances pour un motif. La méthode **findall()** et la métode **finditer()**
- La fonction **findall()** renvoie toutes les correspondances de l'expression régulière passée en argument sous forme d'une liste de chaînes.
- La chaîne est analysée de la gauche vers la droite, et les correspondances sont renvoyées dans l'ordre où elles sont trouvées.

Exemple :

```
>>> import re
>>> p = re.compile(r'\d+')
>>> p.findall("Bonjour 11 Aurevoir 22 Hello 333 By 44")
 ['11', '22', '33', '44']
```

Remarque: Le préfixe **r**, qui indique une chaîne brute littérale, est nécessaire dans cet exemple pour reconnaître les séquences d'échappement!

26

Les expressions régulières

- La méthode **finditer()** renvoie une séquence d'instances d'objets correspondances en tant qu'itérateur :

```
iterator = p.finditer('Bonjour 11 Aurevoir 22 Hello 33 By 44 ')
>>> iterator
<callable_iterator object at 0x...>
>>> for match in iterator:
        ... print(match.span())
        ...
(8, 10)
(20, 22)
(29, 31)
(35, 37)
```


27

Les expressions régulières

```
>>> import re
>>> masque1 = "ier" #correspond à ‘ier’
>>> masque2 = "^[0-9]{10}$" #correspond à une chaine de 10 chiffres exactement

>>> r1 = re.compile(masque1)
>>> r2 = re.compile(masque2)

>>> c1 = " Je m’apelle Pierre, hier j’étais … "
>>> c2 = "0636765787"
>>> c3 = "1234567"

>>> print(re.search(r1,c1))
<re.Match object; span=(14,17), match=‘ier’>

>>> print(re.match(r1,c1))
None

>>> print(re.findall(r1,c1))
[‘ier’, ‘ier’]

>>>print(re.search(r2, c2))
<re.Match object; span=(0,10), match=‘0636765787’>

>>>print(re.search(r2, c3))
None
```

28

Les expressions régulières: Exercice

Ecrivez un programme qui permet de tester si un e-mail est valide ou non.

Le programme prend une liste des e-mails et affiche les e-mails valides et non valides.

Si on considère l'exemple suivant:

```
M =["salmam@mail.com", "salmam@mail.ca", "sal_8@mail.com", "@mail.com", "salmam@mailcom"]
```

Le programme doit afficher:

```
Ce mail : salmam@mail.com est valide
Erreur ce mail : salmam@mail.ca est non valide
Ce mail : sal_8@mail.com est valide
Erreur ce mail : @mail.com est non valide
Erreur ce mail : salmam@mailcom est non valide
```